

Statistics for machine-learning: a brief intro

Data-driven engineering course 2026

Machine-learning is part of statistics

In machine-learning, the goal is to exploit data to:

- extract physical insights,
- model physical (or non-physical) processes,
- estimate input-output relationships.

In general, we assume that the **data** is generated by a **process** that is too **complex** to represent exactly.

We divide the problem in a **simple** part, which we can **represent**, and a **complex** part, which we sweep under the rug of the **uncertainty**.

This problem can be represented implicitly or explicitly.

Example: regression

In **regression**, we want to **estimate** the functional **relationship** between the **inputs** x and the **observations** y .

The simplest way to represent this is to write:

$$y = f(x)$$

We generally accept that we will not obtain a perfect predictions of the observations because:

- the observations contain **noise**,
- we do **not** have **enough information** on the system,
- we are choosing a **simple model**.

We can make explicit the **uncertainty** in the model by writing:

$$y = f(x) + \epsilon$$

Where ϵ is a sample from some probability distribution, which we want to model.

Fitting distributions

Usually we think of **machine-learning** (especially regression) as **fitting functions** to match the data that we have collected.

In the **statistical** representation, we want to **fit** probability **distributions** to our data.

This is beneficial because:

- it allows us to have a measure of the **confidence** in our prediction,
- it improves the **robustness** of the model,
- it **reduces** the number of training **samples**.

Glossary

- A probability distribution $P(X \in E)$ assigns a probability to outcomes.
- A random variable X is a variable that represents a random process.
- An event E is a subset of the sample space.
- The sample space Ω is the set of all possible outcomes.
- Independent and identically distributed (iid) refers to random variables that are independent between each others and are distributed in the same way.
- A discrete random variable is a variable X with a finite number of possible outcomes, which has an associated probability mass function $p_{\theta}(x_i)$ such that $\sum_{x \in \Omega} p_{\theta}(x_i) = 1$
- A continuous random variable is a variable X with an infinite number of possible outcomes, which has an associated probability density function $f_{\theta}(x)$ such that $\int_{\Omega} f_{\theta}(x) dx = 1$

For example, if X is the weather, Ω is {rainy, cloudy, foggy, sunny, etc.} and E could be {rainy or cloudy}.

The symbol θ in the pmf and pdf stands to indicate that the probability function is parametrized in some way.

Glossary

- The expected value of a probability distribution is average value weighted by the probability:

$$\mathbb{E}[p_\theta] = \sum_{x \in \Omega} x_i p_\theta(x_i)$$
$$\mathbb{E}[f_\theta] = \int_{\Omega} x f_\theta(x) dx$$

- The sample mean is the arithmetic average from the collected sample $X_1, \dots, X_n$:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

- Law of large numbers states that the sample mean converges to the expected value when the number of samples tends to infinity:

$$\bar{X}_n \rightarrow \mathbb{E}[p_\theta] \text{ for } n \rightarrow \infty$$

The idea “garbage in, garbage out” for a machine learning model can be translated as the fact that the data needs to be representative of of true underlining distribution to be useful for learning.

Example

My random process is flipping a fair coin, meaning that I expect to see 50% of the times heads (1) and 50% times tails (0).

The sample space is $(0,1)$ and the mathematical distribution that represents this process is a Bernoulli distribution $p_\theta(X) = \text{Ber}_\theta(X) = \theta^X(1 - \theta)^{1-X}$ where $\theta = 0.5$.

The expected value of the distribution is $\mathbb{E}[p_\theta(X)] = \sum_{\Omega}^n X_i \theta^{X_i} (1 - \theta)^{1-X_i} = \theta = 0.5$

If we collect some random data $X_1, \dots, X_n \sim \text{Ber}_\theta(X)$ for $n = 20$ we should see something like: $[0,1,0,1,1,1,0,1,0,1,1,1,0,0,0,1,0,1]$, with an associated sample mean $\bar{X}_n = 0.55$.

How do we compute the probability

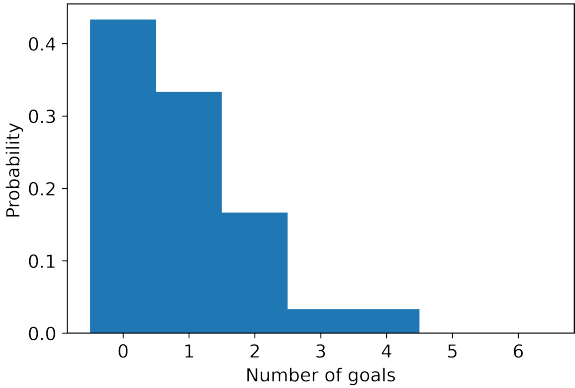
Using a frequentist approach, we can count the number of outcomes, divided by the total number of outcomes.

Example: we want to compute the probability of a player scoring in a football match.

Collect the data: [2,0,1,3,1,1,0,0,1,2,0,2,1,0,0,0,0,2,0,1,0,1,0,1,1,1,4,0,2,0]

Count the outcomes:

outcomes	0	1	2	3	4	5	6 or more
probability	0.43	0.33	0.17	0.03	0.03	0	0



This probability distribution is parametrized by 6 parameters. How do we compare it with a different distribution?

Can we model it using a standard distribution

If possible, we want to model the random process using a canonical distribution.

This let us:

- reduce the number of parameters,
- reduce the data that we need to represent the distribution.

Can we model it using a standard distribution

If possible, we want to model the random process using a canonical distribution.

This let us:

- reduce the number of parameters,
- reduce the data that we need to represent the distribution.

There are different distributions that can be used to model physical systems depending on the context:

- Bernoulli: $p(x; \theta) = \theta^x(1 - \theta)^{1-x} \quad \theta \in [0,1],$
- Binomial: $p(x; \theta, n) = \frac{n!}{x!(n-x)!} \theta^x(1 - \theta)^{n-x} \quad \theta \in [0,1],$
- Poisson: $p(x; \lambda) = \frac{\lambda^x e^{-\lambda}}{x!} \quad \lambda \in [0, \infty]$
- Gaussian: $f(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right) \quad \mu \in \mathbb{R}, \sigma^2 \in [0, \infty]$

You can find a list of important standard distributions here: <https://distribution-explorer.github.io/index.html>

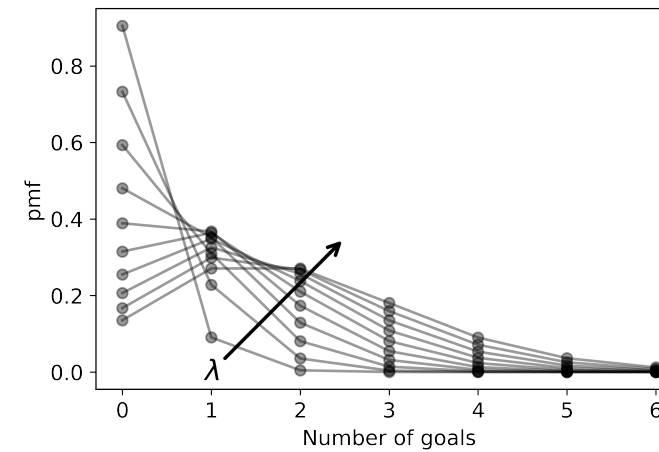
Can we model it using a standard distribution

In our example, we can use the Poisson distribution:

$$p_{\lambda}(x) = \frac{\lambda^x e^{-\lambda}}{x!}$$

which tells us the probability of observing x number of goals for a match.

The parameter λ controls the shape of the distribution.
The higher λ , the more goals I will expect on average.



Can we model it using a standard distribution

Now the problem becomes to find the correct λ that fits the data I have.

First, we need a measure of similarity between distributions and we use the Kullback-Leibler (KL) divergence:

$$\text{KL}(P_\theta, P_{\theta'}) = \sum_{x_i \in \Omega} p_\theta(x_i) \log\left(\frac{p_\theta(x_i)}{p_{\theta'}(x_i)}\right) \quad \text{for discrete probabilities}$$

$$\text{KL}(P_\theta, P_{\theta'}) = \int_{\Omega} f_\theta(x) \log\left(\frac{f_\theta(x)}{f_{\theta'}(x)}\right) dx \quad \text{for continuous probabilities}$$

Some properties of the KL divergence:

- $\text{KL}(P_\theta, P_{\theta'}) \neq \text{KL}(P_{\theta'}, P_\theta)$
- $\text{KL}(P_\theta, P_{\theta'}) \geq 0$
- if $\text{KL}(P_\theta, P_{\theta'}) = 0$ then $P_\theta = P_{\theta'}$

It is called divergence and not “distance” because it is not commutative. However, it possesses some crucial properties to estimate the difference between two probabilities.

Maximum likelihood estimation

We can write the KL divergence in terms of the expectation:

$$\text{KL}(P_{\theta^*}, P_{\theta}) = \mathbb{E}_{\theta^*}[\log(\frac{p_{\theta^*}(X)}{p_{\theta}(X)})] = \mathbb{E}_{\theta^*}[\log(p_{\theta^*}(X))] - \mathbb{E}_{\theta^*}[\log(p_{\theta}(X))]$$

where θ^* is our target (fixed) and θ is our variable.

This means that $\text{KL}(P_{\theta^*}, P_{\theta}) = C - \mathbb{E}_{\theta^*}[\log(p_{\theta}(X))]$, and also we can estimate the expected value:

$$\hat{\text{KL}}(P_{\theta^*}, P_{\theta}) = C - \frac{1}{n} \sum_{i=1}^n \log(p_{\theta}(X_i))$$

To get the two probabilities close, we want to minimize the divergence:

$$\hat{\theta} = \arg \min_{\theta} \hat{\text{KL}}(P_{\theta^*}, P_{\theta}) = \arg \max_{\theta} \prod_{i=1}^n p_{\theta}(X_i) = \arg \max_{\theta} L(X_1, \dots, X_n | \theta)$$

For the “law of the unconscious statistician” $E[g(X)] = \sum(g(x) p(x))$

Intuitively, we want to find the best parameter theta, such that the probability of seeing the outcomes that I have collected is the highest.

Example on the Poisson distribution

The probability of Poisson is $p_\lambda(X) = \frac{\lambda^X e^{-\lambda}}{X!}$

The likelihood is then $L(X_1, \dots, X_n | \lambda) = \prod_{i=1}^n \frac{\lambda^{X_i} e^{-\lambda}}{X_i!} = \frac{\lambda^{\sum_{i=1}^n X_i} e^{-n\lambda}}{X_1! \dots X_n!}$ and

$$\log L = \sum_{i=1}^n \frac{\log(\lambda^{X_i} e^{-\lambda})}{\log(X_i!)} = -\log(X_1! \dots X_n!) + \log(\lambda) \sum_{i=1}^n X_i - n\lambda$$

To find the maximum likelihood estimator $\hat{\lambda}$ we put the derivative of the log likelihood at zero:

$$\frac{\partial \log L}{\partial \lambda} = \frac{\partial(-\log(X_1! \dots X_n!) + \log \hat{\lambda} \sum_{i=1}^n X_i - n\hat{\lambda})}{\partial \lambda} = 0$$
$$\hat{\lambda} = \frac{1}{n} \sum_{i=1}^n X_i = \bar{X}_n$$

The sample average is the maximum likelihood estimator for the Poisson distribution.

Taking the logarithm of the likelihood is done to simplify the computation. Since the logarithm is a monotonic function, the $\arg \max L$ is equal to the $\arg \max \log(L)$.

Let's see if it makes sense

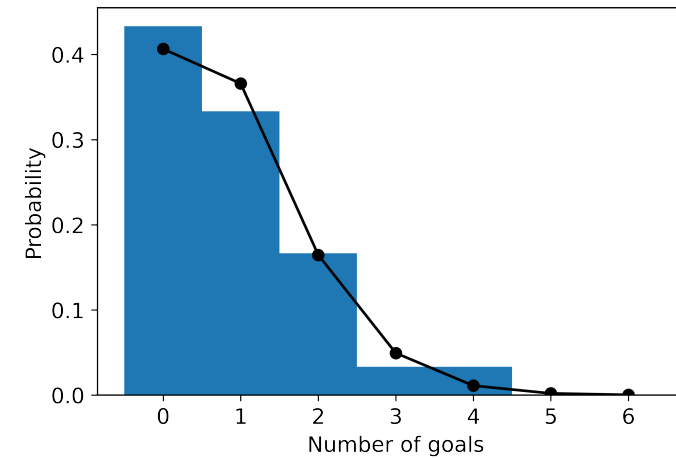
Our dataset was:

[2,0,1,3,1,1,0,0,1,2,0,2,1,0,0,0,0,2,0,1,0,1,0,1,1,1,4,0,2,0]

The sample average is equal to:

$$\frac{1}{n} \sum_{i=1}^n X_i = \frac{1}{30}(2 + 0 + 1 + \dots + 0) = 0.9 = \hat{\lambda}$$

If we compare the histogram of the data with the Poisson distribution with $\lambda = 0.9$ we obtain a pretty good match.



Another one

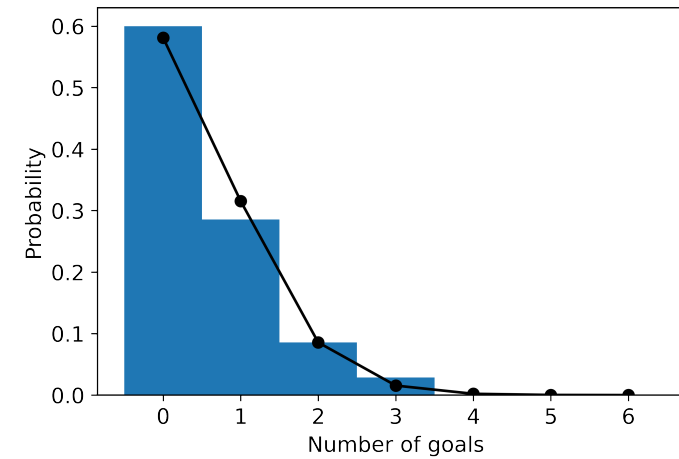
The data is:

[0,0,1,1,1,2,0,0,0,0,2,0,0,1,0,0,0,0,0,0,1,1,2,0,0,0,1,0,0,3,1,0,0,1,1]

The sample average is equal to:

$$\frac{1}{n} \sum_{i=1}^n X_i = 0.54 = \hat{\lambda}$$

Even for two completely different players
the modelling holds well.



We have assumed that the goals are iid, which is not the case from a theory point of view but this works well in a lot of cases.

Least-squares regression

We assume that our model is of the form:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon} \quad \mathbf{y}, \boldsymbol{\epsilon} \in \mathbb{R}^n, \mathbf{X} \in \mathbb{R}^{n \times p}$$

The least squared error (LSE) estimator minimizes the l_2 norm difference

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{t}} \|\mathbf{y} - \mathbf{X}\mathbf{t}\|_2^2 = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Then we can write that $\hat{\mathbf{w}} = \mathbf{w} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \boldsymbol{\epsilon}$ and if $\boldsymbol{\epsilon} \sim \mathcal{N}_n(0, \sigma^2 \mathbf{I})$ then

$$\hat{\mathbf{w}} = \mathbf{w} + \mathcal{N}_p(0, \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1})$$

The optimal estimation of $\mathbf{w}$ is found by computing the gradient of the loss and checking where it is equal to zero.

Least-squares regression

We assume that our model is of the form:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon} \quad \mathbf{y}, \boldsymbol{\epsilon} \in \mathbb{R}^n, \mathbf{X} \in \mathbb{R}^{n \times p}$$

The least squared error (LSE) estimator minimizes the l_2 norm difference

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{t}} \|\mathbf{y} - \mathbf{X}\mathbf{t}\|_2^2 = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Then we can write that $\hat{\mathbf{w}} = \mathbf{w} + (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \boldsymbol{\epsilon}$ and if $\boldsymbol{\epsilon} \sim \mathcal{N}_n(0, \sigma^2 \mathbf{I})$ then

$$\hat{\mathbf{w}} = \mathbf{w} + \mathcal{N}_p(0, \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1})$$

Remarks:

- We can find the estimator if $\mathbf{X}^T \mathbf{X}$ is invertible, which is not the case if $p > n$.
- The estimator is unbiased with variance proportional to $(\mathbf{X}^T \mathbf{X})^{-1}$, meaning that features with high variance are more informative.

If $p > n$ the covariance matrix is rank deficient. For example, $p=2$ with $n=1$ (1 point) means that you cannot fit a line for a single point. Unbiased estimator means that the expected value of the estimator is the true operator $\mathbf{w}$.

MLE of least-squares regression

Since $\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon}$ and $\boldsymbol{\epsilon} \sim \mathcal{N}_n(0, \sigma^2 \mathbf{I})$, the probability density of $\mathbf{y}$ is $\mathcal{N}_n(\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$:

$$p(\mathbf{y}) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2}\right)$$

Then the log likelihood is:

$$\log p(\mathbf{y}) = -n/2 \log(2\pi\sigma^2) - \frac{\|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2}$$

so that:

$$\max_{\mathbf{w}} \log p(\mathbf{y}) = \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$$

and we have that the maximum likelihood estimator is equal to the least squares estimator if we assume that the noise is Gaussian.

Some properties

- LSE = MLE: $\hat{\mathbf{w}} \sim \mathcal{N}_p(\mathbf{w}, \sigma^2(\mathbf{X}^T\mathbf{X})^{-1})$
- Prediction error: $\mathbb{E}[\|\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}\|_2^2] = \sigma^2(n - p)$
- Estimator of the variance: $\hat{\sigma}^2 = \frac{1}{n - p} \|\mathbf{y} - \mathbf{X}\hat{\mathbf{w}}\|_2^2$

Some remarks:

- The equality between LSE and MLE holds only in the case of Gaussian noise.
- The prediction error is equal to zero if $n = p$.
- The $n - p$ factor comes from the fact that the noise lives in the space orthogonal to the span of $\mathbf{X}$, which has dimensions $n - p$.

For $n=p$ the problem doesn't make sense (perfect fit)

Bayesian statistics

Up until now we have employed a frequentist approach (collect data and infer the distribution parameter using the MLE).

In Bayesian statistic, we have a two step process:

- We assume some distribution on the parameters,
- We update the distribution by collecting data.

Bayesian statistics

Up until now we have employed a frequentist approach (collect data and infer the distribution parameter using the MLE).

In Bayesian statistic, we have a two step process:

- We assume some distribution on the parameters,
- We update the distribution by collecting data.

Bayes theorem:

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

so if $p(A)$ is my prior (distribution of the parameters) and $p(B|A)$ is the probability of the data conditioned on the parameters, then we obtain $p(A|B)$ the probability of the parameters conditioned on the data

Bayesian statistics

For continuous probability distributions:

- $\pi(\theta)$ is the prior on the parameter θ ,
- $p(X_1, \dots, X_n | \theta)$ is the likelihood, the probability of the data given the parameter θ ,
- $p(X_1, \dots, X_n) = \int_{\Theta} p(X_1, \dots, X_n | \theta) \pi(\theta) d\theta$ is the marginal likelihood, or the probability distribution of the data itself,
- $p(\theta | X_1, \dots, X_n)$ is the posterior distribution.

The marginal likelihood (or evidence) is generally the hardest part to compute because it involves an integral that is often intractable. However, there exist a number of techniques to compute its approximation.

Example on Poisson distribution

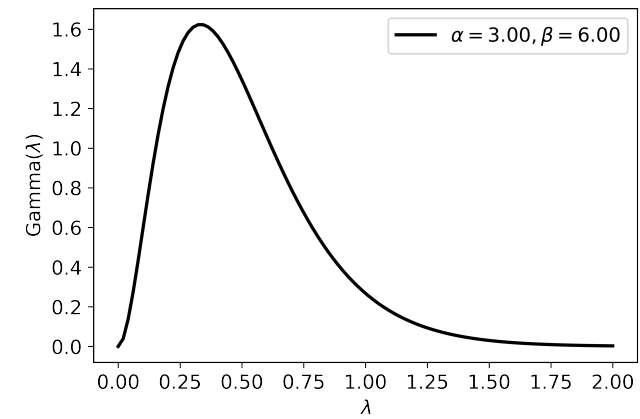
Let's consider our example on the distributions of goals, but this time we put a prior distribution on the parameter λ :

- Prior: $\pi(\lambda) = \text{Gamma}(\lambda, \alpha, \beta) = \frac{\beta^\alpha \lambda^{\alpha-1} e^{-\beta\lambda}}{\Gamma(\alpha)}$
- Likelihood: $p(X_1, \dots, X_n | \lambda) = \frac{\lambda^{\sum_{i=1}^n X_i} e^{-n\lambda}}{X_1! \dots X_n!}$
- Marginal likelihood: $p(X_1, \dots, X_n) = \int_{\Theta} \frac{\lambda^{\sum_{i=1}^n X_i} e^{-n\lambda}}{X_1! \dots X_n!} \frac{\beta^\alpha \lambda^{\alpha-1} e^{-\beta\lambda}}{\Gamma(\alpha)} d\lambda$
- Posterior: $p(\lambda | X_1, \dots, X_n) = \text{Gamma}(\lambda, \alpha + \sum_{i=1}^n X_i, \beta + n)$

The Gamma distribution is the conjugated prior of the Poisson distribution. This means that the posterior distribution is guaranteed to be a Gamma distribution with certain parameters. This simplifies a lot the computation but unfortunately this is true only for a limited set of cases.

Example on Poisson distribution

Prior: $\pi(\lambda) = \text{Gamma}(\lambda, \alpha = 3, \beta = 6)$

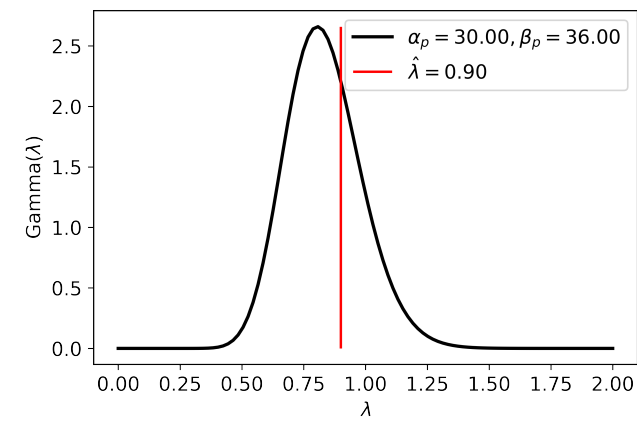


Example on Poisson distribution

Prior: $\pi(\lambda) = \text{Gamma}(\lambda, \alpha = 3, \beta = 6)$

Posterior:

$p(\lambda | X_1, \dots, X_n) = \text{Gamma}(\lambda, \alpha = 30, \beta = 36)$



Example on Poisson distribution

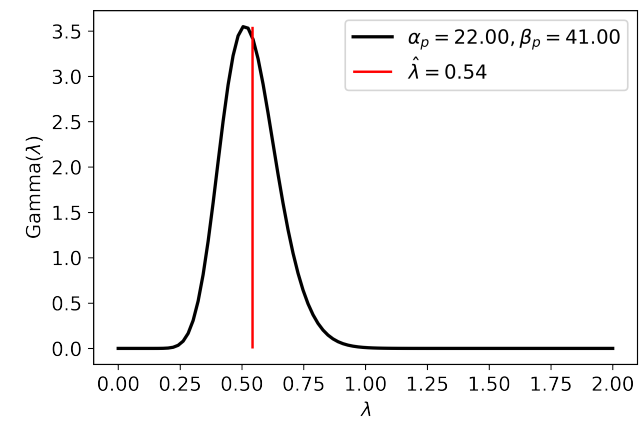
Prior: $\pi(\lambda) = \text{Gamma}(\lambda, \alpha = 3, \beta = 6)$

Posterior:

$$p(\lambda | X_1, \dots, X_n) = \text{Gamma}(\lambda, \alpha = 30, \beta = 36)$$

Posterior:

$$p(\lambda | X_1, \dots, X_n) = \text{Gamma}(\lambda, \alpha = 22, \beta = 41)$$

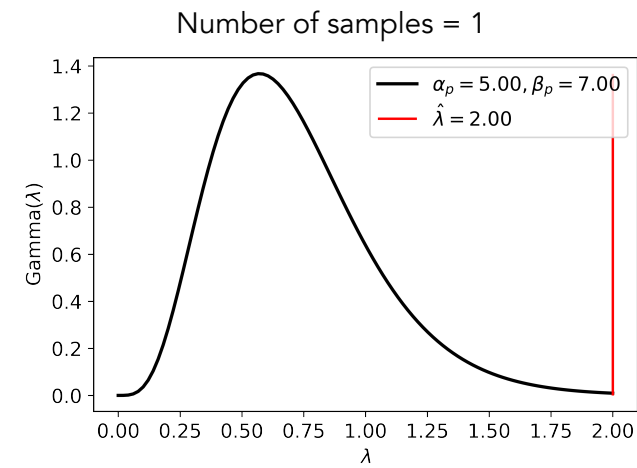


Convergence to the mean

Putting a prior on the distribution parameters let us be more conservative with the data that we need to estimate the parameters.

In general, Bayesian methods are more efficient (if we use a good prior).

The prior is also a good place where we can put physical information about our process.

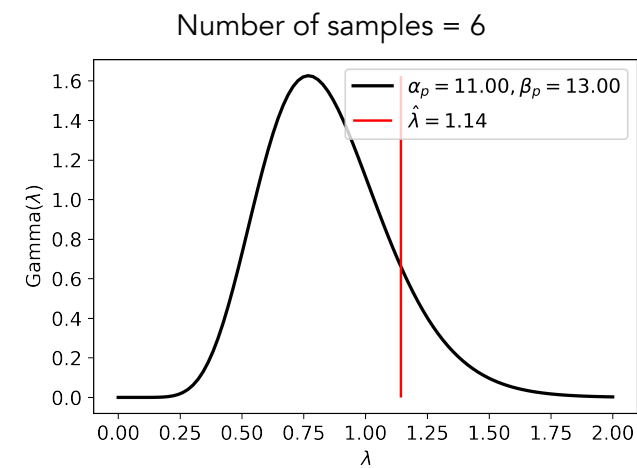


Convergence to the mean

Putting a prior on the distribution parameters let us be more conservative with the data that we need to estimate the parameters.

In general, Bayesian methods are more efficient (if we use a good prior).

The prior is also a good place where we can put physical information about our process.

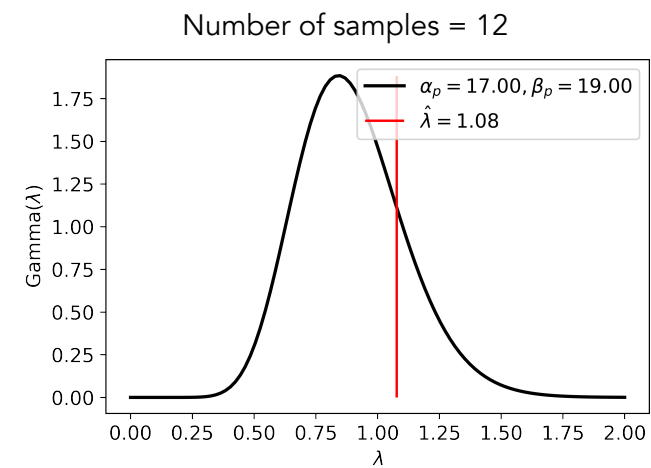


Convergence to the mean

Putting a prior on the distribution parameters let us be more conservative with the data that we need to estimate the parameters.

In general, Bayesian methods are more efficient (if we use a good prior).

The prior is also a good place where we can put physical information about our process.

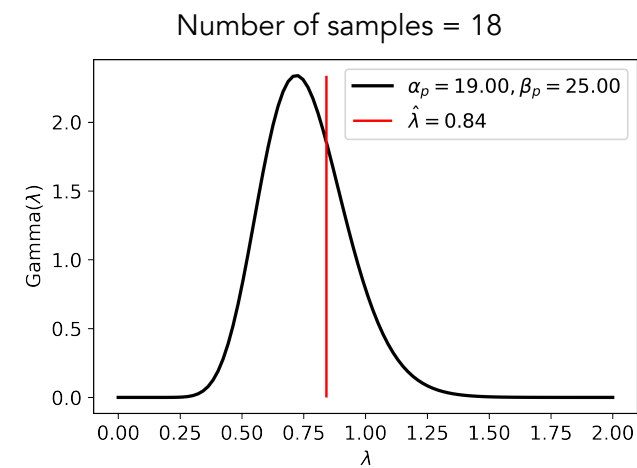


Convergence to the mean

Putting a prior on the distribution parameters let us be more conservative with the data that we need to estimate the parameters.

In general, Bayesian methods are more efficient (if we use a good prior).

The prior is also a good place where we can put physical information about our process.

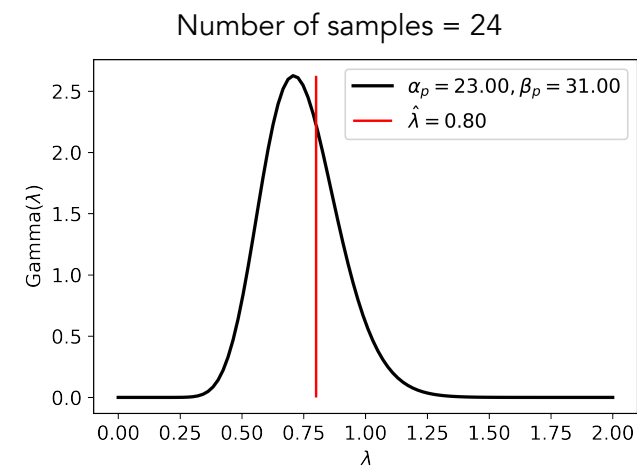


Convergence to the mean

Putting a prior on the distribution parameters let us be more conservative with the data that we need to estimate the parameters.

In general, Bayesian methods are more efficient (if we use a good prior).

The prior is also a good place where we can put physical information about our process.



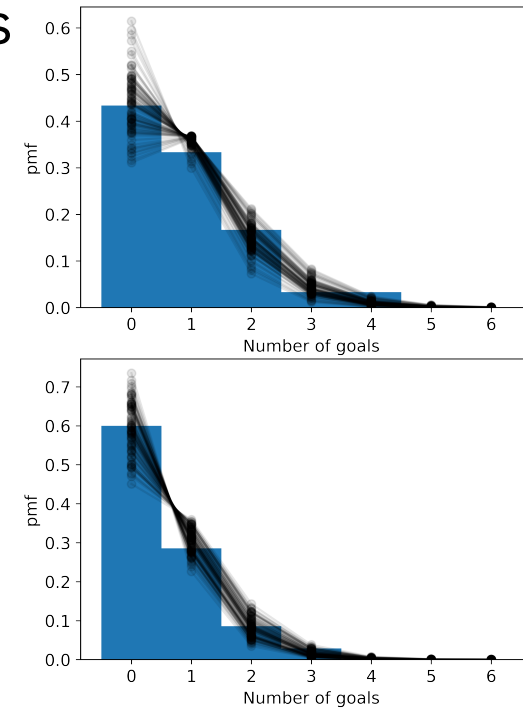
Distribution of distributions

Now that we have a posterior distribution on λ , we can sample from it:

$$\lambda_i \sim p(\lambda | X_1, \dots, X_n)$$

$$p(x, \lambda_i) = \frac{\lambda^x e^{-\lambda_i}}{x!} \quad i = 1, \dots, m$$

This information is more powerful than knowing only the MLE, because it provides a distribution of distributions.



If we do not have any idea

$$p(\theta | X_1, \dots, X_n) = \frac{p(X_1, \dots, X_n | \theta) \pi(\theta)}{\int_{\Theta} p(X_1, \dots, X_n | \theta) \pi(\theta) d\theta}$$

The equation still works, we just assume that $\pi(\theta)$ is equal to one.

This way we do not put any weights on θ (non informative prior) but we can still obtain a posterior distribution which can give us an idea of the uncertainty estimation.

Technically, this prior is not a true distribution (improper prior), but we can still use it to find the posterior as the ratio between the likelihood and its integration.

Gaussian model

$$p(\theta | X_1, \dots, X_n) = \frac{p(X_1, \dots, X_n | \theta) \pi(\theta)}{\int_{\Theta} p(X_1, \dots, X_n | \theta) \pi(\theta) d\theta}$$

For example, the posterior of a Gaussian likelihood with uninformative prior:

$$X_1, \dots, X_n \sim \mathcal{N}(\theta, 1)$$

$$\pi(\theta) = 1$$

$$p(X_1, \dots, X_n | \theta) = \frac{1}{\sqrt{2\pi}^n} \exp\left(-\frac{1}{2} \sum_{i=1}^n (X_i - \theta)^2\right)$$

$$p(\theta | X_1, \dots, X_n) = \mathcal{N}(\bar{X}_n, \frac{1}{n})$$

This result makes sense because, when increasing the number of samples, the mean converges to the true mean, and the variance tends to zero.

Bayesian treatment of OLS

Let's consider again the linear regression model with Gaussian noise:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

We already know what is the likelihood of this model:

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{\|\mathbf{Y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2}\right) = \mathcal{N}(\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$$

We assume that the prior is a Gaussian distribution $p(\mathbf{w} | \mathbf{X}) = \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_p)$

Bayesian treatment of OLS

Let's consider again the linear regression model with Gaussian noise:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + \boldsymbol{\epsilon} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

We already know what is the likelihood of this model:

$$p(\mathbf{y} | \mathbf{X}, \mathbf{w}) = \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{\|\mathbf{Y} - \mathbf{X}\mathbf{w}\|_2^2}{2\sigma^2}\right) = \mathcal{N}(\mathbf{X}\mathbf{w}, \sigma^2 \mathbf{I})$$

We assume that the prior is a Gaussian distribution $p(\mathbf{w} | \mathbf{X}) = \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_p)$

Applying Bayes we obtain that the posterior is equal to:

$$p(\mathbf{w} | \mathbf{y}, \mathbf{X}) = \frac{p(\mathbf{y} | \mathbf{X}, \mathbf{w})p(\mathbf{w} | \mathbf{X})}{p(\mathbf{y} | \mathbf{X})}$$

Turns out that $p(\mathbf{w} | \mathbf{y}, \mathbf{X}) = \mathcal{N}(\bar{\mathbf{w}}, \text{cov}(\mathbf{w}))$ with

$$\bar{\mathbf{w}} = \frac{1}{\sigma^2} \left(\frac{1}{\sigma^2} \mathbf{X}^T \mathbf{X} + \boldsymbol{\Sigma}_p^{-1} \right)^{-1} \mathbf{X}^T \mathbf{y}$$

$$\text{cov}(\mathbf{w}) = \left(\frac{1}{\sigma^2} \mathbf{X}^T \mathbf{X} + \boldsymbol{\Sigma}_p^{-1} \right)^{-1}$$

Note that the bayesian converges to the frequentist when $\text{Sigma}_p \rightarrow \text{inf}$, meaning that the Bayesian OLS and the frequentist OLS give the same result when the prior is completely uninformative.

An example

We want to predict the heating demand h of a building as a function of the temperature difference δT and socio-economic indicator s :

$$h = w_1 \cdot \delta T + w_2 \cdot s + \epsilon$$

h are the **observations** while $\mathbf{z} = \{w_1, w_2, \epsilon\}$ are the **latent variables**.

Bayes' theorem tells us that:

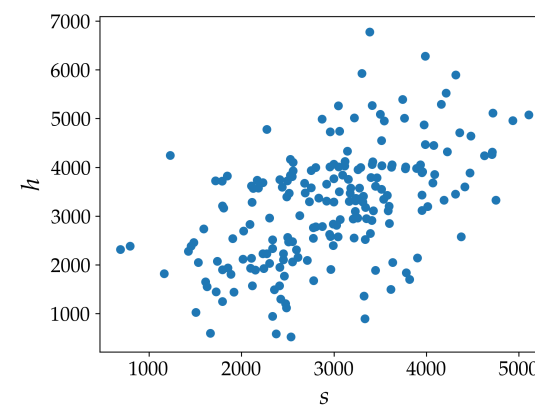
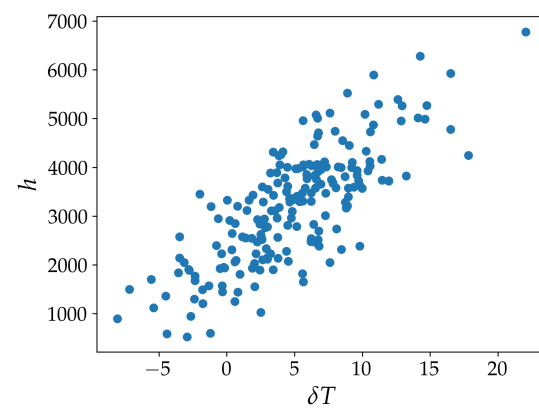
$$p(\mathbf{z} | h) = \frac{p(h | \mathbf{z})p(\mathbf{z})}{p(h)}$$

So the goal is to start from a **prior** over the latent variable then add the **observations** to **improve** our **knowledge** over $\mathbf{z}$ (the posterior).

An example

I have built a synthetic dataset considering $w_1 = 200$, $w_2 = 0.75$ and $\epsilon \sim \mathcal{N}(0, 250)$

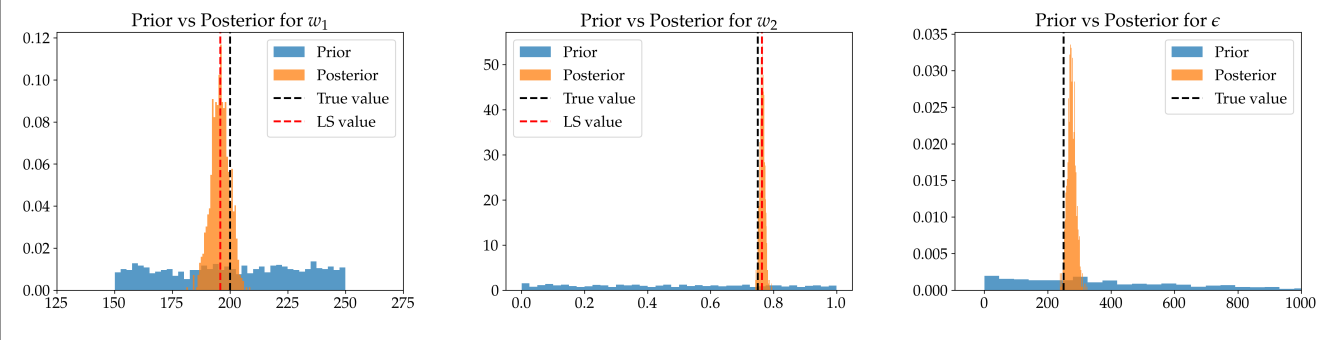
$$h = w_1 \cdot \delta T + w_2 \cdot s + \epsilon$$



An example

The priors are setup as $p(w_1) = \mathcal{U}(150, 250)$, $p(w_2) = \mathcal{U}(0, 1)$ and $p(\epsilon) = \mathcal{N}_{1/2}(0, 500)$

The least squares solution is computed as $p^{LS}, \alpha^{LS} = \arg \min_{\mathbf{w}} ||w_1 \cdot \delta T + w_2 \cdot s - h||_2^2$



Predictive posterior

In a deterministic model, the prediction is estimated by considering only the “optimal” latent variables:

$$h^* = \mathbf{z}^o \mathbf{x}^*$$

Problems:

- **no** information on the **uncertainty** of h^* ,
- risk of **overfitting**.

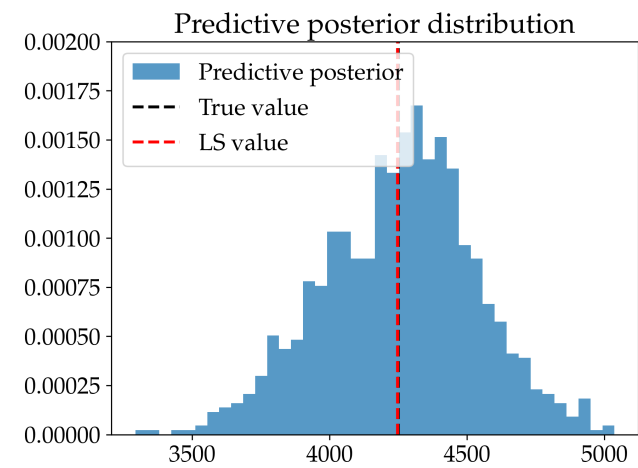
In a Bayesian model the probability of new values h^* given the observed ones is obtained by:

$$p(h^* | h) = \int_{\Theta} p(h^* | \mathbf{z}) p(\mathbf{z} | h) d\mathbf{z}$$

Instead of considering only the best estimation $\mathbf{z}^o$, by **marginalising** over the distribution of $p(\mathbf{z} | h)$ we obtain a more **realistic** predictive **distribution**.

Simple example

Let's say we want to predict the heating demand for a case where $\delta T = 10$ and the household salary is $s = 3000$



Stochastic variational inference

To find the posterior distribution, we have applied the Bayes' theorem:

$$p(\mathbf{z} | h) = \frac{p(h | \mathbf{z})p(\mathbf{z})}{p(h)}$$

Problem: when we use uniform distribution as prior and Gaussian distribution as likelihood, the evidence does not have a closed form solution

$$p(h) = \int_{\Theta} p(h | \mathbf{z})p(\mathbf{z}) d\mathbf{z} = ?$$

Without knowing the evidence, we cannot compute the posterior.

Stochastic variational inference

In most cases, the **posterior** and the **evidence** do not have an analytic representation.

The problem is:

- we **cannot train** the parameters,
- we **cannot represent** the posterior.

One solution is to **substitute** the **true** distribution $p(h, \mathbf{z})$ with an approximate distribution $q_\phi(\mathbf{z})$.

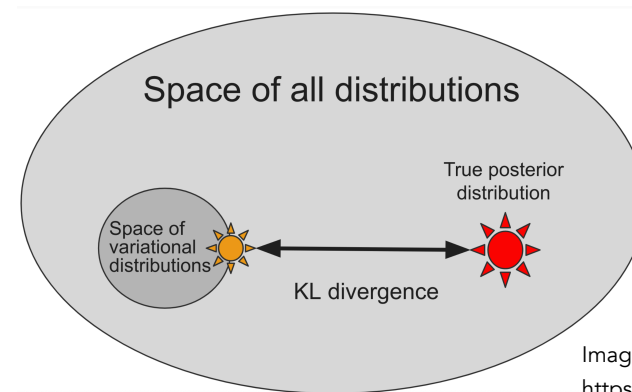


Image taken from Pyro

https://pyro.ai/examples/intro_long.html

Stochastic variational inference

The idea is: instead of computing $p(\mathbf{z} | h)$, we find an approximated probability $q_{\phi}(\mathbf{z})$ such that:

$$q_{\phi^*}(\mathbf{z}) \approx p(\mathbf{z} | h)$$

The problem shift to find the optimal parameters ϕ^* such that we obtain a good approximation.

How do we do that? We find a metric to minimize:

$$\text{ELBO}(\phi) = \mathbb{E}_{q_{\phi}(\mathbf{z})}[\log p(\mathbf{z}, h) - \log q_{\phi}(\mathbf{z})]$$

The ELBO has two properties:

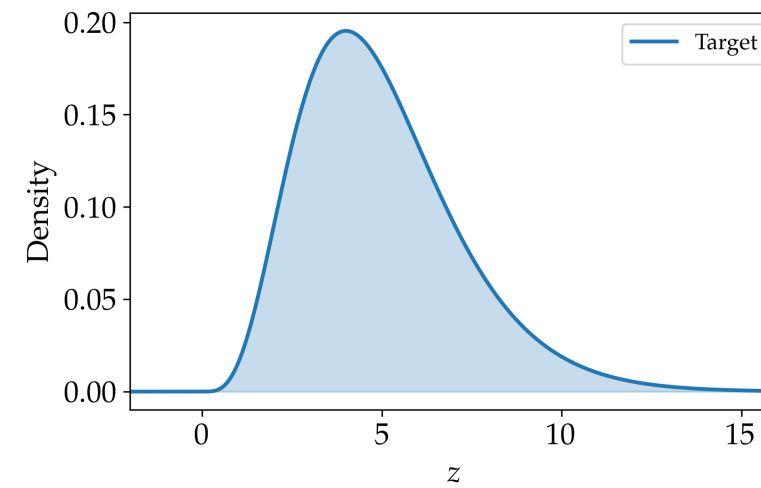
- $\log p(h) \geq \text{ELBO}$ -> if we **increase ELBO** we reduce the distance with the evidence,
- $\log p(h) - \text{ELBO} = \text{KL}(q_{\phi}(\mathbf{z}) || p(\mathbf{z} | h))$ -> if we push **ELBO higher**, the **distance** between $q_{\phi}(\mathbf{z})$ and $p(\mathbf{z} | h)$ will **decrease**.

By maximising the ELBO we obtain:

- the **optimal parameters** ϕ^* ,
- the **approximated posterior** $q_{\phi^*}(\mathbf{z}) \approx p(\mathbf{z} | h)$

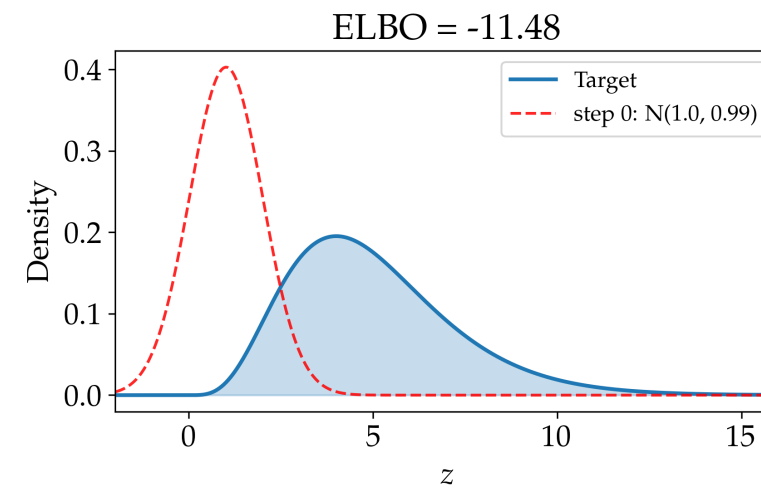
Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



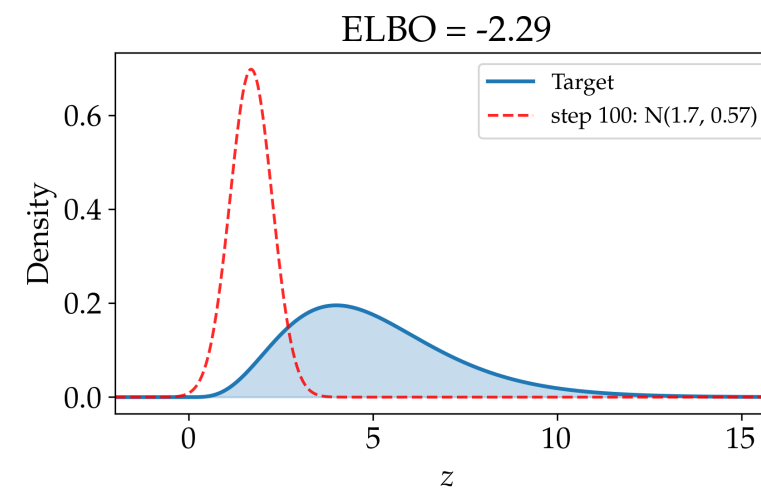
Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



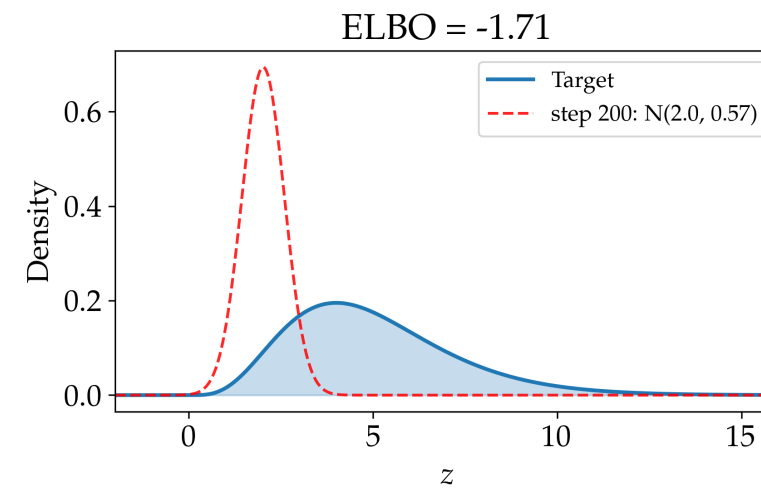
Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



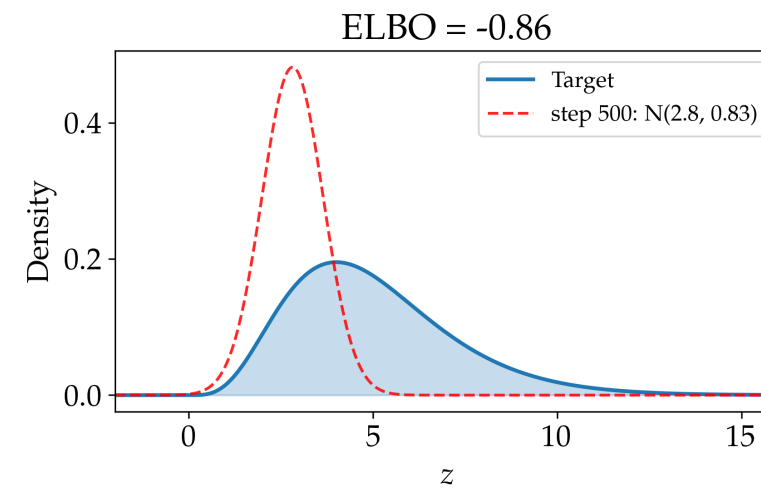
Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



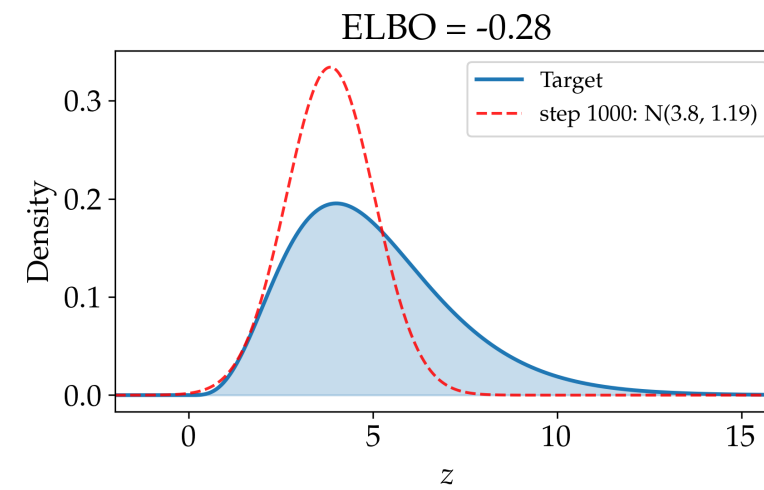
Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



Example

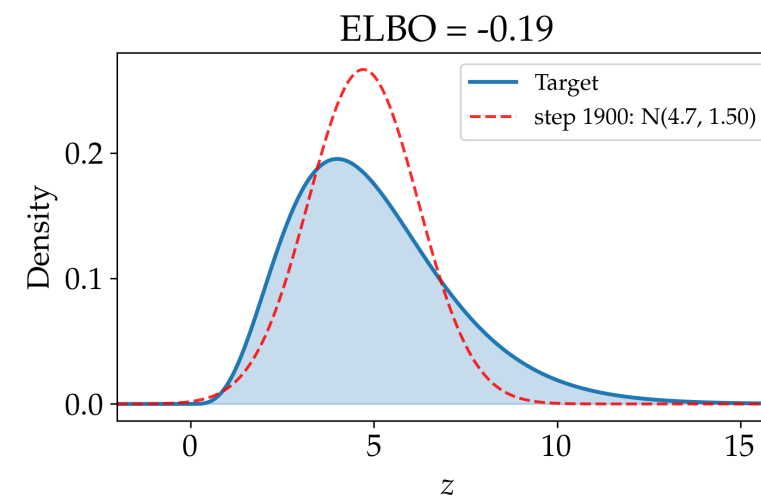
We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.



Example

We want to approximate a target probability distribution with a Gaussian distribution.
The goal is to set the parameters of the Gaussian distribution so they fit the target.

The ELBO increases when the approximating distribution gets closer to the target distribution.



Gaussian process regression

In the case of linear regression we have assumed that we can model the observed data as a sum of a deterministic part ($\mathbf{X}\mathbf{w}$) plus a random part (ϵ).

Gaussian process regression

In the case of linear regression we have assumed that we can model the observed data as a sum of a deterministic part ($\mathbf{X}\mathbf{w}$) plus a random part (ϵ).

In GPR instead we directly assume that the entire function is a sample from a Gaussian process:

$$f(x) \sim GP(\mu(x), k(x, x'))$$

Similar to the normal distribution, a Gaussian process is a distribution over functions, which is parametrized by a mean function $\mu(x)$ and a covariance (or kernel) function $k(x, x')$.

Gaussian process regression

In the case of linear regression we have assumed that we can model the observed data as a sum of a deterministic part ($\mathbf{X}\mathbf{w}$) plus a random part (ϵ).

In GPR instead we directly assume that the entire function is a sample from a Gaussian process:

$$f(x) \sim GP(\mu(x), k(x, x'))$$

Similar to the normal distribution, a Gaussian process is a distribution over functions, which is parametrized by a mean function $\mu(x)$ and a covariance (or kernel) function $k(x, x')$.

In practise, we approximate functions with arrays, and so we approximate the Gaussian process with a multivariate normal distribution:

$$\mathbf{f} \sim \mathcal{N}(\boldsymbol{\mu}, \mathbf{K})$$

Aside: properties of multivariate Gaussians

Let's imagine that we have a probability distribution for two Gaussian random variables $\mathbf{f}_1$ and $\mathbf{f}_2$.

The **joint** probability is:

$$p(\mathbf{f}_1, \mathbf{f}_2) = \mathcal{N}(\boldsymbol{\mu}, \mathbf{K}) \quad \boldsymbol{\mu} = [\boldsymbol{\mu}_1, \boldsymbol{\mu}_2] \quad \mathbf{K} = \begin{bmatrix} \mathbf{K}_{1,1} & \mathbf{K}_{1,2} \\ \mathbf{K}_{1,2}^T & \mathbf{K}_{2,2} \end{bmatrix}$$

The **marginal** probability is obtained by integrating away the other variable:

$$p(\mathbf{f}_1) = \int p(\mathbf{f}_1, \mathbf{f}_2) d\mathbf{f}_2 = \mathcal{N}(\boldsymbol{\mu}_1, \mathbf{K}_{1,1})$$

The **conditional** probability is obtained by dividing the joint probability with the marginal probability:

$$p(\mathbf{f}_1 | \mathbf{f}_2) = \frac{p(\mathbf{f}_1, \mathbf{f}_2)}{p(\mathbf{f}_2)} = \mathcal{N}(\boldsymbol{\mu}_1 + \mathbf{K}_{1,2} \mathbf{K}_{2,2}^{-1} (\mathbf{f}_2 - \boldsymbol{\mu}_2), \mathbf{K}_{1,1} - \mathbf{K}_{1,2} \mathbf{K}_{2,2}^{-1} \mathbf{K}_{1,2}^T)$$

Aside: properties of multivariate Gaussians

The product of a multivariate Gaussian and a matrix is still a multivariate Gaussian:

$$\mathbf{A}\mathbf{f}_1 \sim \mathcal{N}(\mathbf{A}\boldsymbol{\mu}_1, \mathbf{A}\mathbf{K}_{1,1}\mathbf{A}^T)$$

The product of two multivariate Gaussians is still a multivariate Gaussian:

$$p(\mathbf{f}_1)p(\mathbf{f}_2) = \mathcal{N}(\boldsymbol{\mu}_1, \mathbf{K}_1)\mathcal{N}(\boldsymbol{\mu}_2, \mathbf{K}_2) = c^{-1}\mathcal{N}\left(\frac{\mathbf{K}_1^{-1}\boldsymbol{\mu}_1 + \mathbf{K}_2^{-1}\boldsymbol{\mu}_2}{\mathbf{K}_1^{-1} + \mathbf{K}_2^{-1}}, (\mathbf{K}_1^{-1} + \mathbf{K}_2^{-1})^{-1}\right)$$

$$c^{-1} = (2\pi)^{-D/2} |\mathbf{K}_1 + \mathbf{K}_2|^{-1/2} \exp(-1/2(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^T(\mathbf{K}_1 + \mathbf{K}_2)^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2))$$

GPR is a Bayesian method

We collect n observations $\{\mathbf{x}_i, y_i\}_n$ and we assemble them into the input matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ and the array of observations $\mathbf{y} \in \mathbb{R}^n$.

GPR is a Bayesian method

We collect n observations $\{\mathbf{x}_i, y_i\}_n$ and we assemble them into the input matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ and the array of observations $\mathbf{y} \in \mathbb{R}^n$.

We start by selecting our prior, which is usually a normal distribution with zero mean:

$$p(\mathbf{f} | \mathbf{X}, \phi) = \mathcal{N}(\mathbf{0}, \mathbf{K}(\phi))$$

To compute the covariance matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$, we use a kernel function $\mathbf{K}_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j; \phi)$.

The kernel function measures the similarity between points and it is controlled by the hyperparameter ϕ .

GPR is a Bayesian method

We collect n observations $\{\mathbf{x}_i, y_i\}_n$ and we assemble them into the input matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ and the array of observations $\mathbf{y} \in \mathbb{R}^n$.

We start by selecting our prior, which is usually a normal distribution with zero mean:

$$p(\mathbf{f} | \mathbf{X}, \phi) = \mathcal{N}(\mathbf{0}, \mathbf{K}(\phi))$$

To compute the covariance matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$, we use a kernel function $\mathbf{K}_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j; \phi)$.

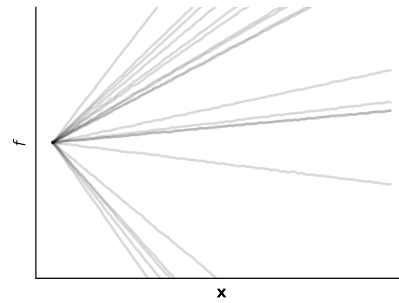
The kernel function measures the similarity between points and it is controlled by the hyperparameter ϕ .

The simplest kernel function is the linear kernel, which assumes that the covariance between points is proportional to the inner product:

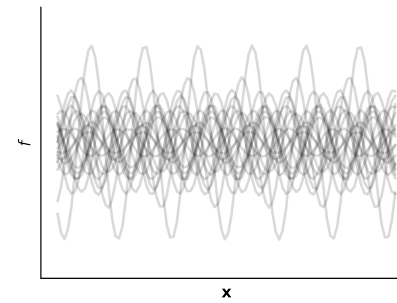
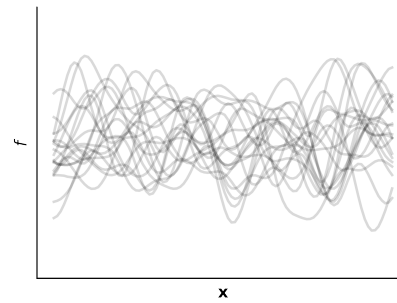
$$k_l(\mathbf{x}_i, \mathbf{x}_j; \phi) = \phi \mathbf{x}_i^T \mathbf{x}_j$$

Some kernels

Let's plot some samples from the prior with different kernels:



$$k_l(\mathbf{x}_i, \mathbf{x}_j; \phi) = \phi \mathbf{x}_i^T \mathbf{x}_j$$



$$k_c(\mathbf{x}_i, \mathbf{x}_j; \phi) = \cos(\pi \|\mathbf{x}_i - \mathbf{x}_j\|_2 / \phi)$$

$$k_{RBF}(\mathbf{x}_i, \mathbf{x}_j; \phi) = \exp\left(-\frac{(\mathbf{x}_i - \mathbf{x}_j)^T(\mathbf{x}_i - \mathbf{x}_j)}{2\phi^2}\right)$$

How to chose the kernel

The **choice** of the kernels depends on the **type** of **regression** problem.

How to chose the kernel

The **choice** of the kernels depends on the **type** of **regression** problem.

The **RBF** is generally a good choice because it is very **flexible** and it can adapt to different types of models. But, since the covariance drops exponentially it works **very bad** in **extrapolation**.

How to chose the kernel

The **choice** of the kernels depends on the **type** of **regression** problem.

The **RBF** is generally a good choice because it is very **flexible** and it can adapt to different types of models. But, since the covariance drops exponentially it works **very bad** in **extrapolation**.

The **kernel** can be used also to include **physical information** on the system. For example, if we know that the system is periodic we can use a mixture of cosine kernels.

How to chose the kernel

The **choice** of the kernels depends on the **type** of **regression** problem.

The **RBF** is generally a good choice because it is very **flexible** and it can adapt to different types of models. But, since the covariance drops exponentially it works **very bad** in **extrapolation**.

The **kernel** can be used also to include **physical information** on the system. For example, if we know that the system is periodic we can use a mixture of cosine kernels.

A **kernel** can be also created by **composing** different **kernel** functions (adding a linear kernel with a periodic kernel for example).

Compute the posterior

We want to compute the posterior $p(\mathbf{f}|\mathbf{y}, \mathbf{X}, \phi)$

So we need:

- the likelihood $p(\mathbf{y}|\mathbf{f}, \mathbf{X}, \phi)$
- the marginal likelihood $p(\mathbf{y}|\mathbf{X}, \phi) = \int p(\mathbf{y}|\mathbf{f}, \mathbf{X}, \phi)p(\mathbf{f}|\mathbf{X}, \phi)d\mathbf{f}$

Compute the posterior

We want to compute the posterior $p(\mathbf{f}|\mathbf{y}, \mathbf{X}, \phi)$

So we need:

- the likelihood $p(\mathbf{y}|\mathbf{f}, \mathbf{X}, \phi)$
- the marginal likelihood $p(\mathbf{y}|\mathbf{X}, \phi) = \int p(\mathbf{y}|\mathbf{f}, \mathbf{X}, \phi)p(\mathbf{f}|\mathbf{X}, \phi)d\mathbf{f}$

The likelihood maps the model to the observation, i.e. is the noise:

$$p(\mathbf{y}|\mathbf{f}, \mathbf{X}, \phi) = \mathcal{N}(0, \sigma^2 \mathbf{I})$$

The **marginal likelihood** represents the **probability** of observing the **data given** my choice of kernel **hyperparameters**.

The marginal likelihood

In general, it is very hard to compute directly the marginal likelihood.

However, if both the likelihood and the prior are Gaussian, then the marginal likelihood has a closed-form solution:

$$\log p(\mathbf{y} | \mathbf{X}, \phi) = \log \int p(\mathbf{y} | \mathbf{f}, \mathbf{X}, \phi) p(\mathbf{f} | \mathbf{X}, \phi) d\mathbf{f} = -\frac{1}{2} \mathbf{y}^T (\mathbf{K}(\phi) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}(\phi) + \sigma^2 \mathbf{I}| - n/2 \log 2\pi$$

The marginal likelihood

In general, it is very hard to compute directly the marginal likelihood.

However, if both the likelihood and the prior are Gaussian, then the marginal likelihood has a closed-form solution:

$$\log p(\mathbf{y} | \mathbf{X}, \phi) = \log \int p(\mathbf{y} | \mathbf{f}, \mathbf{X}, \phi) p(\mathbf{f} | \mathbf{X}, \phi) d\mathbf{f} = -\frac{1}{2} \mathbf{y}^T (\mathbf{K}(\phi) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}(\phi) + \sigma^2 \mathbf{I}| - n/2 \log 2\pi$$

To find the optimal kernel parameters, we want to maximize $\log p(\mathbf{y} | \mathbf{X}, \phi)$, which is equivalent to minimize $-\log p(\mathbf{y} | \mathbf{X}, \phi)$.

The marginal likelihood

In general, it is very hard to compute directly the marginal likelihood.

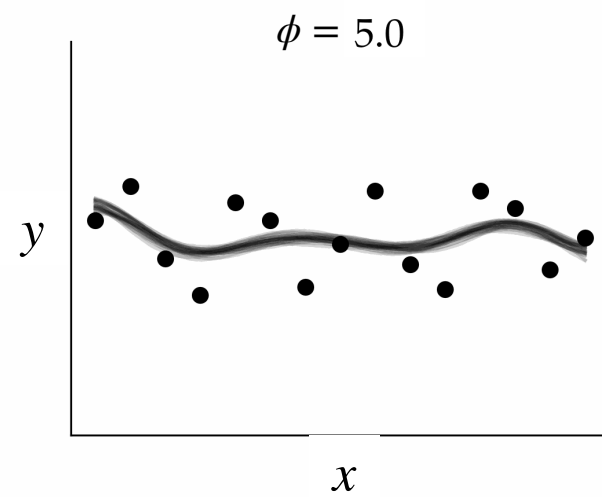
However, if both the likelihood and the prior are Gaussian, then the marginal likelihood has a closed-form solution:

$$\log p(\mathbf{y} | \mathbf{X}, \phi) = \log \int p(\mathbf{y} | \mathbf{f}, \mathbf{X}, \phi) p(\mathbf{f} | \mathbf{X}, \phi) d\mathbf{f} = -\frac{1}{2} \mathbf{y}^T (\mathbf{K}(\phi) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}(\phi) + \sigma^2 \mathbf{I}| - n/2 \log 2\pi$$

To find the optimal kernel parameters, we want to maximize $\log p(\mathbf{y} | \mathbf{X}, \phi)$, which is equivalent to minimize $-\log p(\mathbf{y} | \mathbf{X}, \phi)$.

The negative log marginal likelihood is not a convex function, and so we need to employ methods (such as gradient descent) to find its minimum.

The marginal likelihood



$$\log p(\mathbf{y} | \mathbf{X}, \phi) = -\frac{1}{2} \mathbf{y}^T (\mathbf{K}(\phi) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}(\phi) + \sigma^2 \mathbf{I}| - n/2 \log 2\pi = -24.14$$

The predictive posterior

For new points $\mathbf{X}^*$, we want to find the predictive posterior:

$$p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o)$$

The predictive posterior

For new points $\mathbf{X}^*$, we want to find the predictive posterior:

$$p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o)$$

This is the probability distribution in new points, conditioned on the observations $\mathbf{y}$, thus:

$$p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o) = p(\mathbf{f}^*, \mathbf{y} | \mathbf{X}, \mathbf{X}^*, \phi^o) / p(\mathbf{y} | \mathbf{X}, \phi^o)$$

Joint probability $p(\mathbf{f}^*, \mathbf{y} | \mathbf{X}, \mathbf{X}^*, \phi^o) = \mathcal{N}(0, \begin{bmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{K}_* \\ \mathbf{K}_* & \mathbf{K}_{**} \end{bmatrix})$

Marginal probability $p(\mathbf{y} | \mathbf{X}, \phi^o) = \mathcal{N}(0, \mathbf{K} + \sigma^2 \mathbf{I})$

The predictive posterior

For new points $\mathbf{X}^*$, we want to find the predictive posterior:

$$p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o)$$

This is the probability distribution in new points, conditioned on the observations $\mathbf{y}$, thus:

$$p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o) = p(\mathbf{f}^*, \mathbf{y} | \mathbf{X}, \mathbf{X}^*, \phi^o) / p(\mathbf{y} | \mathbf{X}, \phi^o)$$

$$\text{Joint probability } p(\mathbf{f}^*, \mathbf{y} | \mathbf{X}, \mathbf{X}^*, \phi^o) = \mathcal{N}\left(0, \begin{bmatrix} \mathbf{K} + \sigma^2 \mathbf{I} & \mathbf{K}_* \\ \mathbf{K}_* & \mathbf{K}_{**} \end{bmatrix}\right)$$

$$\text{Marginal probability } p(\mathbf{y} | \mathbf{X}, \phi^o) = \mathcal{N}(0, \mathbf{K} + \sigma^2 \mathbf{I})$$

$$\text{Predictive posterior } p(\mathbf{f}^* | \mathbf{y}, \mathbf{X}, \mathbf{X}^*, \phi^o) = \mathcal{N}(\bar{\mathbf{f}}^*, \text{cov}(\mathbf{f}^*))$$

$$\bar{\mathbf{f}}^* = \mathbf{K}_* (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{y}$$

$$\text{cov}(\mathbf{f}^*) = \mathbf{K}_{**} - \mathbf{K}_* (\mathbf{K} + \sigma^2 \mathbf{I})^{-1} \mathbf{K}_*$$

An example

We generate data from the model $y = x \sin(x) + \mathcal{N}(0,1)$ $x \in [\pi, 3\pi]$

Since the function to regress is nonlinear, we opt for the RBF kernel with a scaling parameter:

$$k_{RBF}(\mathbf{x}_i, \mathbf{x}_j; \phi) = s \exp\left(-\frac{(\mathbf{x}_i - \mathbf{x}_j)^T(\mathbf{x}_i - \mathbf{x}_j)}{2\phi^2}\right)$$

We train the model by minimizing the negative log marginal likelihood to find the set of optimal parameters $[s, \phi, \sigma] = [37.2, 1.53, 0.9]$.

